Venus - Risk Reward Receiver

Security Assessment

CertiK Assessed on Dec 29th, 2025

CertiK Assessed on Dec 29th, 2025

Venus - Risk Reward Receiver

The security assessment was prepared by CertiK.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Lending	Binance Smart Chain (BSC)	Manual Review, Static Analysis

LANGUAGE	TIMELINE
Solidity	Preliminary comments published on 12/18/2025 Final report published on 12/29/2025

Vulnerability Summary

22	21	1	0	0	0
Total Findings	Resolved	Timelock	Partially Resolved	Acknowledged	Declined

1	Centralization	1 Timelock	Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.
0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
0	Major		Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.
1	Medium	1 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
3	Minor	3 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
17	Informational	17 Resolved	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | VENUS - RISK REWARD RECEIVER

CODEBASE | VENUS - RISK REWARD RECEIVER

Repository

https://github.com/VenusProtocol/governance-contracts

Commit

Base: 675e568581df93da1cf08f0d5e8b1168e94e7f37
Update1: 7f20b3fc63b7bbbef3c0351d9467a0ee63a53ccb

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | VENUS - RISK REWARD RECEIVER

This audit was conducted for Venus to evaluate the security and correctness of the smart contracts associated with the Venus - Risk Reward Receiver project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

OVERVIEW | VENUS - RISK REWARD RECEIVER

This audit concerns the changes made in the in scope files outlined in the following PR:

- PR-163

Note that any centralization risks present in the existing codebase before these PRs were not considered in this audit and only those added in these PRs are addressed in the audit. We recommend all users carefully review the centralization risks, much of which can be found in our previous audits, which can be found here: https://skynet.certik.com/projects/venus.

PR-163

PR-163 introduces a risk stewarding framework allowing some risk parameter updates to by-pass full governance proposals when those updates are evaluated as *safe* (for instance, with low change ratio). The PR specifies three stewarding flows:

- Immediate execution for safe updates, via a call to the adequate stewarding contract (if any) which checks that the parameters satisfy governance-specified safety conditions.
- Timelocked Execution for high-delta and sensitive updates, run (or rejected) by approved executors after the timelock delay.
- Cross-chain update, which always require the approval of a whitelisted executor before a stewarding contract is called.

DEPENDENCIES | VENUS - RISK REWARD RECEIVER

Third Party Dependencies

The protocol is serving as the underlying entity to interact with third party protocols. The third parties that the contracts interact with are:

- LayerZero
- ERC20 Tokens
- Oracles

The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets. Moreover, updates to the state of a project contract that are dependent on the read of the state of external third party contracts may make the project vulnerable to read-only reentrancy. In addition, upgrades of third parties can possibly create severe impacts, such as increasing fees of third parties, migrating to new LP pools, etc.

Recommendations

We recommend constantly monitoring the third parties involved to mitigate any side effects that may occur when unexpected changes are introduced, as well as vetting any third party contracts used to ensure no external calls can be made before updates to its state.

FINDINGS | VENUS - RISK REWARD RECEIVER

22	0	1	0	1	3	17
Total Findings	Critical	Centralization	Major	Medium	Minor	Informational

This report has been prepared for Venus to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 22 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
VRR-09	**Centralization Related Risks**	**Centralization**	**Centralization**	**1h Timelock**
VRR-10	Missing Checks	Logical Issue	Medium	Resolved
VRR-13	Same Default For All Chains	Logical Issue	Minor	Resolved
VRR-14	Missing Zero Address Validation	Volatile Code	Minor	Resolved
VRR-15	`CollateralFactorsRiskSteward` May Allow Unsupported Update Type	Logical Issue	Minor	Resolved
VRR-04	Discussion On Use Of `OAppUpgradeable`	Inconsistency	Informational	Resolved
VRR-06	Discussion On Fixed Remote Delay	Design Issue	Informational	Resolved
VRR-07	Discussion On LayerZero Fees	Design Issue	Informational	Resolved
VRR-08	Discussion On Pausing Conventions	Logical Issue	Informational	Resolved
VRR-11	Potential Issues With `CORE_POOL_COMPTROLLER` Set On Remote Chains	Volatile Code	Informational	Resolved
VRR-16	`delete` Keyword Can Be Used	Inconsistency	Informational	Resolved

ID	Title	Category	Severity	Status
VRR-17	Missing Redundancy Check	Inconsistency	Informational	Resolved
VRR-18	Safe Delta Check Allows Redundant Setting	Logical Issue	Informational	Resolved
VRR-19	Empty Function Body	Inconsistency	Informational	Resolved
VRR-20	Inconsistent Expiration Logic For Remote Updates	Inconsistency	Informational	Resolved
VRR-21	Not All Contracts Disable Renouncing Ownership	Inconsistency	Informational	Resolved
VRR-22	`allUpdateTypes` Can Cause Gas Issues If Large Amount Of Update Types Are Added	Logical Issue	Informational	Resolved
VRR-23	Incomplete/Missing Comments	Inconsistency	Informational	Resolved
VRR-24	Documentation Mismatch	Inconsistency	Informational	Resolved
VRR-27	Misleading Name	Coding Issue	Informational	Resolved
VRR-28	Discussion On Remote Update Debounces	Design Issue	Informational	Resolved
VRR-29	Discussion On Market Uniqueness Across Chains	Logical Issue	Informational	Resolved

VRR-09 | Centralization Related Risks

Category	Severity	Location	Status
Centralization	Centralization	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 131~132, 174~175; RiskSteward/DestinationStewardReceiver.sol (Base): 268~269, 305~306, 330~331, 353~354, 399~400, 495; RiskSteward/IRMRiskSteward.sol (Base): 118~119; RiskSteward/MarketCapsRiskSteward.sol (Base): 127~128; RiskSteward/RiskOracle.sol (Base): 130, 147, 164, 186, 222, 247; RiskSteward/RiskStewardReceiver.sol (Base): 130~131, 145~146, 167~168, 218~219, 243~244, 300~301, 316~317, 339~340, 471~472; RiskSteward/DestinationStewardReceiver.sol (Update1): 181	1h Timelock

Description

Note that any centralization risks present in the existing codebase before the PR's in scope of this audit were not considered. Only those added to the in-scope PRs are addressed. We recommend all users carefully review the centralization risks, much of which can be found in our previous audits, which can be found here: https://skynet.certik.com/projects/venus.

CollateralFactorsRiskSteward

In the contract `CollateralFactorsRiskSteward`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the function:

- `setSafeDeltaBps()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set the safe delta to 1 to disable the contract effectively.
- Set the safe delta to maximum (100 %) to bypass full security controls on associated risk config updates, allowing setting collateral factors or liquidation thresholds very high or very low.

In the contract `CollateralFactorsRiskSteward`, the role `RISK_STEWARD_RECEIVER` has authority over the function:

- `applyUpdate()`

Any compromise to the `RISK_STEWARD_RECEIVER` role may allow the hacker to take advantage of this authority and do the following:

- Execute risk config updates without full security controls.

DestinationStewardReceiver

In the contract `DestinationStewardReceiver`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setRiskParameterConfig()`
- `setConfigActive()`
- `setWhitelistedExecutor()`
- `setRemoteDelay()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Specify malicious update configs, e.g. by setting extremely debounce periods for some actions.
- Disable legitimate update configs.
- Whitelist malicious accounts and unwhitelist legitimate ones.
- Set the remote delay to a short value to prevent proper review before execution.
-

In the contract `DestinationStewardReceiver`, the role `Owner` has authority over the following functions:

- `transferOwnership()` Any compromise to the `Owner` account may allow the hacker to take advantage of this authority and do the following:
- Transfer ownership to another malicious entity.

In the contract `DestinationStewardReceiver`, any address added as a `whitelistedExecutor` has authority over the following functions:

- `executeUpdate()`
- `rejectUpdate()`

Any compromise to accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Reject legitimate updates.

IRMRiskSteward

In the contract `IRMRiskSteward`, the role `RISK_STEWARD_RECEIVER` has authority over the function:

- `applyUpdate()`

Any compromise to the `RISK_STEWARD_RECEIVER` role may allow the hacker to take advantage of this authority and do the following:

- Execute risk config updates without full security controls.

MarketCapsRiskSteward

In the contract `MarketCapsRiskSteward`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant

addresses the privilege to call the function:

- `setSafeDeltaBps()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Set the safe delta to 1 to disable the contract effectively.
- Set the safe delta to maximum (100 %) to bypass full security controls on associated risk config updates.

In the contract `MarketCapsRiskSteward`, the role `RISK_STEWARD_RECEIVER` has authority over the function:

- `applyUpdate()`

Any compromise to the `RISK_STEWARD_RECEIVER` role may allow the hacker to take advantage of this authority and do the following:

- Execute risk config updates without full security controls.

RiskOracle

In the contract `RiskOracle`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `addAuthorizedSender()`
- `removeAuthorizedSender()`
- `addUpdateType()`
- `setUpdateTypeActive()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Add or remove authorized senders either allowing or disallowing them to call the privileged functions protected by the `onlyAuthorized` modifier.
- Add and make a new update type active.
- Make an update type inactive or active.

In the contract `RiskOracle`, any address added as an `authorizedSenders` can grant addresses the privilege to call the following functions:

- `publishRiskParameterUpdate()`
- `publishBulkRiskParameterUpdates()`

Any compromise to accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Publish malicious risk parameter updates.

RiskStewardReceiver

In the contract `RiskStewardReceiver`, the role `DEFAULT_ADMIN_ROLE` of the `AccessControlManager` can grant addresses the privilege to call the following functions:

- `setPaused()`
- `setRiskParameterConfig()`
- `setConfigActive()`
- `setWhitelistedExecutor()`

Any compromise to the `DEFAULT_ADMIN_ROLE` or accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Pause or unpause the contract, thus disabling some of its intended functionalities.
- Clog the contract with risk parameters configs.
- Create configs pointing to malicious risk stewards.
- Deactivate legitimate configs.
- Add or remove whitelisted executors.

In the contract `RiskStewardReceiver`, the role `Owner` has authority over the following functions:

- `sweepNative()`
- `transferOwnership()` Any compromise to the `Owner` account may allow the hacker to take advantage of this authority and do the following:
- Steal all native tokens from the contract.
- Transfer ownership to another malicious entity.

In the contract `RiskStewardReceiver`, any address added as a `whitelistedExecutor` has authority over the following functions:

- `executeRegisteredUpdate()`
- `rejectUpdate()`
- `resendRemoteUpdate()`

Any compromise to accounts granted this privilege may allow the hacker to take advantage of this authority and do the following:

- Reject legitimate updates.
- Continually resend remote updates, potentially creating issues such as DoS on the destination chain.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend

centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

[Venus, 12/24/2025]: The ownership of all contracts will be transferred to the standard Venus Normal Timelock. ACM permissions will be assigned to specific timelocks based on the required level of control, normal, fast-track, or critical, and a guardian will be designated for critical setters. All permission assignments will be executed through a VIP and approved via Venus governance, accompanied by a community post to ensure transparency to users.

Governance-related addresses are here: https://docs-v4.venus.io/deployed-contracts/governance.

VRR-10 | Missing Checks

Category	Severity	Location	Status
Logical Issue	Medium	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 206; RiskSteward/DestinationStewardReceiver.sol (Base): 244, 272~274; RiskSteward/RiskOracle.sol (Base): 326; RiskSteward/RiskStewardReceiver.sol (Base): 108, 176, 306, 630~633	Resolved

Description

RiskOracle

- The function `_publishUpdate()` does not check that the input market is supported by the core pool or isolated pools.

RiskStewardReceiver

- The `constructor()` does not check that the input `layerZeroLzEid_` is nonzero.
- The function `_validateRegisterUpdate()`, does not check that if an update is subjected to a timelock that the registered updates unlock time will not exceed the updates expiration time. This allows for an update to be registered with a timelock that will only allow it to be executed after the expiration time and thus never be able to be executed.
- The function `executeRegisteredUpdate()`, does not check if the configs `riskSteward` is the same as when the update was registered. It is possible that a config is updated to use a different `riskSteward` than when the update is registered and timelocked, causing the update to use a different `riskSteward` to execute the update.
- The function `setRiskParameterConfig()` does not revert if `bytes(newUpdateType).length > 64`. This makes it inconsistent with the function `RiskOracle.addUpdateType()`.

DestinationStewardReceiver

- The function `setRiskParameterConfig()` does not revert if `bytes(newUpdateType).length > 64`. This makes it inconsistent with the function `RiskOracle.addUpdateType()`.
- The `constructor()` does not check that the input `layerZeroLzEid_` is nonzero.

CollateralFactorsRiskSteward

- The function `_updateCollateralFactors()` does not check the length of `newValue` is 64 bytes. Note that `_decodeAbiEncodedTwoUint256()` could be used.

Recommendation

We recommend adding the missing checks above or ensuring that the privileged entities make this check prior to calling the functions.

Alleviation

[Venus, 12/24/2025]: The Risk Oracle receives updates for multiple chains, including remote ones where we can't reliably check if a market is listed. Since `_publishUpdate()` is only called by whitelisted risk-parameter providers who already validate inputs, an extra listing check isn't needed. Any invalid market would be rejected later by the comptroller or the market contracts.

We don't expect to update the `RiskSteward` contract soon. If needed, the team will review all impacts before making changes.

[CertiK, 12/24/2025]: Considering the comments above and the recommended changes that were made in the following commits, we consider this finding resolved.

- b6ba91bc641f619a01033f9319832c490df1f181;
- 9967888d8d09c518448ba3143f1f32e7366888c2;
- 5b47b4ff7acdb24d0dece0862c5290a320187ed3.

VRR-13 | Same Default For All Chains

Category	Severity	Location	Status
Logical Issue	Minor	RiskSteward/RiskStewardReceiver.sol (Base): 548	Resolved

Description

The contract uses by default `OptionsBuilder.newOptions().addExecutorLzReceiveOption(1_000_000, 0)` to specify 1,000,000 gas for executing LayerZero messages on the destination chain. This value is arbitrary and may not be sufficient on all chains at all times as opcode costs may change over time and blockchain. Conversely, it may be higher than necessary on chains with lower gas costs, resulting in overpayment.

Recommendation

We recommend using chain-specific default gas settings to account for the differing gas costs across different remote chains.

Alleviation

[Venus, 12/24/2025]: We are using ~2x the gas needed on Ethereum, and since we're only supporting EVM chains, this gives us enough buffer everywhere. Extra gas isn't wasted because LayerZero refunds whatever isn't used. We'll also re-check gas requirements across all supported chains before going live.

VRR-14 | Missing Zero Address Validation

Category	Severity	Location	Status
Volatile Code	Minor	RiskSteward/RiskStewardReceiver.sol (Base): 104	Resolved

Description

RiskStewardReceiver

- The `constructor()` does not check that the input `endpoint_` is not the zero address.

Recommendation

We recommend adding a zero address check for the passed in address value to prevent unexpected errors.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 9967888d8d09c518448ba3143f1f32e7366888c2.

VRR-15 | `CollateralFactorsRiskSteward` May Allow Unsupported Update Type

Category	Severity	Location	Status
Logical Issue	Minor	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 147~148	Resolved

Description

The `isSafeForDirectExecution()` function returns false if update.poolId is not zero, but does not first verify that `update.updateTypeKey` equals `COLLATERAL_FACTORS_KEY`. This omission means updates with unsupported `updateTypeKey` values might still proceed through the registration process, potentially leading to unintended behavior or acceptance of invalid updates.

Recommendation

We recommend ensuring that this function reverts in any case if the `updateTypeKey` is not equal to the `COLLATERAL_FACTORS_KEY`.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 9a461161f436b80891bc994b1f85a8eea36b931c.

VRR-04 | Discussion On Use Of `OAppUpgradeable`

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/DestinationStewardReceiver.sol (Base): 20; RiskSteward/RiskStewardReceiver.sol (Base): 22	Resolved

Description

The contracts `DestinationStewardReceiver` and `RiskStewardReceiver` inherit from `OAppUpgradeable`, which includes both sending and receiving message capabilities. However, `DestinationStewardReceiver` is designed to only receive messages, and `RiskStewardReceiver` is designed to only send messages. Each contract could inherit from the more specific `OAppReceiverUpgradeable` or `OAppSenderUpgradeable` contracts. This would more accurately reflect their functional scope and reduce unnecessary inherited functionality. While using the more specific base contracts may require future adjustments if the contracts' roles change, the current design indicates they are only expected to send or receive messages.

Recommendation

We recommend considering the comments above.

Alleviation

[CertiK, 12/26/2025]: The client updated the code to use `OAppReceiverUpgradeable` and `OAppSenderUpgradeable` in commit 9ae58f700d36e593bdbcbc722e047f6bb1c2979c.

VRR-06 | Discussion On Fixed Remote Delay

Category	Severity	Location	Status
Design Issue	Informational	RiskSteward/DestinationStewardReceiver.sol (Base): 20	Resolved

Description

All updates executed on the remote chain currently use a single, fixed remote delay. This approach means that every update type, regardless of its risk profile or purpose, is subject to the same timelock period. Some update types may benefit from a longer or shorter delay to better align with their security or operational requirements. Introducing an adjustable timelock mechanism for each update type can provide more granular control and improve overall protocol flexibility.

Recommendation

We recommend clarifying the design choice to instead use a fixed remote delay.

Alleviation

[CertiK, 12/26/2025]: The client updated the code so that the fixed delay was configurable in commit aedaeb0bad7209ab0c83342f804e3c59043f76cf.

[Venus, 12/29/2025]: We use per update-type delays in the RSR. In the destination contract, we apply a single common timelock because we do not fully trust the bridge. All remote updates are treated as unsafe for direct execution and therefore require a strict, uniform delay.

VRR-07 | Discussion On LayerZero Fees

Category	Severity	Location	Status
Design Issue	Informational	RiskSteward/RiskStewardReceiver.sol (Base): 559	Resolved

Description

The `processUpdate()` function is not marked as payable, so it cannot receive native tokens alongside a function call. As a result, the contract must maintain a sufficient balance of native tokens to cover LayerZero fees consistently. This design choice means the contract must be proactively funded with native tokens to ensure continued operation, rather than allowing the caller to supply the required fees per call. Please confirm that this aligns with the intended operational model for the contract.

Recommendation

We recommend answering the question above.

Alleviation

[Venus, 12/24/2025]: Yes, this is intentional. Since the bridge fee is minimal, we plan to keep the contract pre-funded with enough native tokens to cover LayerZero fees.

VRR-08 | Discussion On Pausing Conventions

Category	Severity	Location	Status
Logical Issue	Informational	RiskSteward/RiskStewardReceiver.sol (Base): 300	Resolved

Description

Currently, only the `processUpdate()` function is restricted when the contract is paused, meaning it cannot be called to register or execute new updates. However, if `processUpdate()` was used to register an update before the contract was paused, that update remains executable through the `executeRegisteredUpdate()` function, even while the contract is paused. It is unclear whether allowing the execution of previously registered updates during a pause is an intended aspect of the contract's pausing logic.

Recommendation

We recommend ensuring this design aligns with system requirements.

Alleviation

[Venus, 12/24/2025]: Yes, this is intended. We pause processUpdate() because it has no caller validation and we want to ensure an effective emergency pause. In contrast, executeRegisteredUpdate() is restricted to whitelisted executors, so its execution is handled and controlled by the Venus team.

VRR-11 Potential Issues With `CORE_POOL_COMPTROLLER` Set On Remote Chains

Category	Severity	Location	Status
Volatile Code	Informational	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 109, 208; RiskSteward/IRMRiskSteward.sol (Base): 35~36, 157	Resolved

Description

The comments above `CORE_POOL_COMPTROLLER` state that it will not be used for remote chain deployments. If this means it will be set to `address(0)`, then we recommend checking if it is set to `address(0)` and explicitly skipping any logic intended to only be executed for the core pool.

Recommendation

We recommend adding a zero-check for the locations cited above to prevent unexpected errors.

Alleviation

[Venus, 12/24/2025]: Yes, CORE_POOL_COMPTROLLER will be set to the zero address for remote chains. The existing logic already checks whether the comptroller address matches before proceeding, and this condition will naturally fail when CORE_POOL_COMPTROLLER is zero. Therefore, the intended behavior is preserved, and we do not require an explicit zero-address check.

VRR-16 | `delete` Keyword Can Be Used

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/RiskOracle.sol (Base): 151	Resolved

Description

In the function `removeAuthorizedSender()`, the authorized sender is removed by explicitly setting `authorizedSenders[sender] = false`. In Solidity, it is clearer and more idiomatic to use the `delete` keyword for this purpose, as it directly resets the mapping value to its default (false for booleans) and communicates intent more clearly to other developers.

Recommendation

We recommend using the `delete` keyword when setting variables to their default.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 0a55a89e97b7aba4cbda1c4f01a9a546724a216a.

VRR-17 | Missing Redundancy Check

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 137; RiskSteward/MarketCapsRiskSteward.sol (Base): 133	Resolved

Description

The function `setSafeDeltaBps()` does not verify whether the provided input value is different from the existing value of `safeDeltaBps`. This can result in unnecessary state changes, leading to redundant transactions and potentially higher gas costs, without altering contract behavior.

Recommendation

We recommend adding a check to ensure the input value is different from the current `safeDeltaBps` value before updating the state. This helps prevent unnecessary state changes and avoids additional gas costs from redundant transactions.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit f4d3fcb5d9b7a8cf7d8bc59df5ac04a6e4cda8b4.

VRR-18 | Safe Delta Check Allows Redundant Setting

Category	Severity	Location	Status
Logical Issue	Informational	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 261; RiskSteward/MarketCapsRiskSteward.sol (Base): 235	Resolved

Description

The function `_isWithinSafeDelta()` permits changes where the difference between the current and new values is zero, allowing redundant updates that set the value to its existing state. This behavior does not impact contract security but may lead to unnecessary transactions and potential misconfigurations, as typically there is no benefit in setting a parameter to its current value.

Recommendation

We recommend including a check to prevent updates when the new value is identical to the current value.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit f4d3fcb5d9b7a8cf7d8bc59df5ac04a6e4cda8b4.

VRR-19 | Empty Function Body

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/RiskStewardReceiver.sol (Base): 707~716	Resolved

Description

The function `_lzReceive()` is overridden as it is necessary in order to have the contract compile. However, it is overridden so with an empty function body, allowing the contract to receive cross chain messages. However, while it will receive the messages, it will do nothing with them, which may cause confusion if there are misconfigurations. Instead of having an empty function body, it could revert, so that any messages accidentally sent to the contract will revert and warn users of some error.

Recommendation

We recommend that the `_lzReceive()` function reverts when called to prevent accidental acceptance of cross-chain messages that are not intended to be processed by this contract. This will ensure that any such messages result in a clear and immediate error, reducing the risk of confusion or silent failures due to misconfiguration. Alternatively, consider inheriting from `OAppSenderUpgradeable` if receiving functionality is not needed.

Alleviation

[CertiK, 12/24/2025]: The client resolved the issue by inheriting from `OAppSenderUpgradeable` in commit 9ae58f700d36e593bdbcbc722e047f6bb1c2979c.

VRR-20 | Inconsistent Expiration Logic For Remote Updates

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/DestinationStewardReceiver.sol (Base): 372~373	Resolved

Description

In contract DestinationStewardReceiver:

1. Function `executeUpdate()` reverts if `update.timestamp + REMOTE_UPDATE_EXPIRATION_TIME < currentTime`, i.e., it requires that `block.timestamp <= update.timestamp + REMOTE_UPDATE_EXPIRATION_TIME` (large inequality)
2. Function `_checkPendingUpdate(uint256 currentRegisteredId)` returns (if no early exit) `current.update.timestamp + REMOTE_UPDATE_EXPIRATION_TIME > block.timestamp`. In other word, it specifies the update as pending if `block.timestamp < update.timestamp + REMOTE_UPDATE_EXPIRATION_TIME` (strict inequality).

This is an inconsistency.

Recommendation

We recommend that the client align the expiration and pending-status conditions across `executeUpdate()` and `_checkPendingUpdate()` by using a single, consistently defined inequality (and possibly, a shared internal helper) to avoid ambiguous or contradictory update state handling.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 9a03fcdf2eabf21327c3a38f290233358087b6fe.

VRR-21 | Not All Contracts Disable Renouncing Ownership

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/DestinationStewardReceiver.sol (Base): 20; RiskSteward/RiskOracle.sol (Base): 13	Resolved

Description

All in-scope contracts inherit from `AccessControlledV8`, which itself inherits `Ownable2StepUpgradeable`. Most of these contracts override and disable the `renounceOwnership()` function to prevent ownership from being renounced. However, `DestinationStewardReceiver` and `RiskOracle` do not disable this function. This inconsistency may result in ownership being unintentionally or unnecessarily renounced in these contracts, potentially leaving them without an owner and administrative controls in place.

Recommendation

We recommend that `renounceOwnership()` is uniformly disabled in all contracts or clarifying the reasoning when it is disabled or not.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commits:

- 631a04f49ab2ee7a8399680679d5ba051eab0859
- 8a8f3e5d22801c5be529b600332dbf86049910a2

VRR-22 `allUpdateTypes` Can Cause Gas Issues If Large Amount Of Update Types Are Added

Category	Severity	Location	Status
Logical Issue	Informational	RiskSteward/RiskOracle.sol (Base): 389~396	Resolved

Description

The `allUpdateTypes` array is used to determine whether a specific update type exists by iterating through every element. If the array of update types grows substantially, operations that involve checking for existence will become increasingly gas-intensive due to linear search costs.

Recommendation

If the number of update types is expected to grow large, then we recommend considering the use of a mapping structure for existence checks.

Alleviation

[Venus, 12/24/2025]: The number of update types is expected to remain very limited.

VRR-23 | Incomplete/Missing Comments

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 173, 197, 231~232; RiskSteward/DestinationStewardReceiver.sol (Base): 69, 238, 264, 328, 540~542; RiskSteward/IRMRiskSteward.sol (Base): 117, 153; RiskSteward/MarketCapsRiskSteward.sol (Base): 144; RiskSteward/RiskOracle.sol (Base): 115, 130, 147, 164, 186, 210~211, 235~237; RiskSteward/RiskStewardReceiver.sol (Base): 359	Resolved

Description

RiskOracle

- The comments above the `initialize()` function state that it throws `ZeroAddressNotAllowed`, however, it would revert with an error message "invalid acess control manager address".
- The comments above the function `publishBulkRiskParameterUpdates()` do not include the `ArrayLengthMismatch` and `ZeroAddressNotAllowed` errors.
- The comments above the function `publishRiskParameterUpdate()` do not include the `ZeroAddressNotAllowed` error.
- The comments above functions that call `_checkAccessAllowed()` do not include a comment about their access control.

RiskStewardReceiver

- The comments above the function `getExecutableUpdates()` state "comptroller The address of the Isolated Pools Comptroller that manages the markets". However, the comptroller could also be the core pool comptroller. In addition, both the core and isolated pools have the same `getAllMarkets()` function so that they can use the same interface, we recommend adding a clarifying comment.

DestinationStewardReceiver

- The comments above `LAYER_ZERO_EID` state it should be the source chain EID, while those in the constructor state it should be the destination chain EID. We recommend ensuring the comments are consistent.
- The comments above `setRiskParameterConfig()` state that it emits an event "with previousTimelock and timelock always emitted as 0", however, no such event parameters exist.
- The comments above `setWhitelistedExecutor()` do not include the `ZeroAddressNotAllowed` error.
- The comments above `_checkPendingUpdate()` do not match its behavior.

CollateralFactorsRiskSteward

- The comments above `_updateCollateralFactors()` do not include the error `SetCollateralFactorFailed()`.
- The comments above `applyUpdate()` do not include the error `SetCollateralFactorFailed()`.

- The comments above `_getCurrentCollateralFactors()` state "For core pool, uses eMode-specific getter which handles poolId == 0 as regular market." This comment is unclear and does not indicate that this function is only called if the `poolId = 0` and as such uses the specific getter `CORE_POOL_COMPTROLLER.markets()` which fetches the values for the the core pool, that is the pool with `poolId = 0`.

IRMRiskSteward

- The comments above `_updateIRM()` do not include the error `SetInterestRateModelFailed`.
- The comments above `applyUpdate()` do not include the error `SetInterestRateModelFailed`.

MarketCapsRiskSteward

- In the function `isSafeForDirectExecution` the `ICorePoolComptroller` interface is used for both the core and isolated pools as they both conform to the same interface for the functions being called. However, there is no clarifying comment in the code explaining this.

Recommendation

We recommend adjusting the NatSpec comments mentioned above.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commits:

- 7f20b3fc63b7bbbef3c0351d9467a0ee63a53ccb;
- d6d20bbb4782c39b456da761c7e1c847f1d23c9d;
- 95b8efc1dcbc10b188a369f55834020e2abe1978.

VRR-24 | Documentation Mismatch

Category	Severity	Location	Status
Inconsistency	Informational	RiskSteward/CollateralFactorsRiskSteward.sol (Base): 19~20	Resolved

Description

The Notion documentation provided by the client features one inconsistency:

- CollateralFactorsRiskSteward is mentioned to handle liquidation incentive updates, but no such functionality is implemented in the contract

Recommendation

We recommend that the client either implement the mentioned functionalities, or remove them from the documentation.

Alleviation

[Venus, 12/24/2025]: We will update the documentation and remove the Liquidation Incentive section, as it is not in scope.

VRR-27 | Misleading Name

Category	Severity	Location	Status
Coding Issue	Informational	RiskSteward/DestinationStewardReceiver.sol (Base): 86	Resolved

Description

The mapping `lastRegisteredUpdate` is misleadingly named, as it stores update identifiers rather than the updates themselves. This ambiguity is reinforced by the function `getLastRegisteredUpdate()`, which is not a direct getter but relies on the `updates` mapping using this identifier.

Recommendation

We recommend that the client rename `lastRegisteredUpdate` to `lastRegisteredUpdateId` (or equivalent) to accurately reflect its purpose and improve code clarity and maintainability.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit a93643d90328526e4c6fe0bb5cd1f5e54b7d8719.

VRR-28 | Discussion On Remote Update Debounces

Category	Severity	Location	Status
Design Issue	Informational	RiskSteward/RiskStewardReceiver.sol (Base): 273	Resolved

Description

Remote updates will have a debounce on the source chain and another debounce on the remote chain. We would like to ensure this multiple debounce is desired.

Recommendation

We recommend clarifying the intended design.

Alleviation

[CertiK, 12/26/2025]: The client updated the code to so that the debounce is only enforced on the destination chain in commit d2ac3936d4f91e082ef11b54fca6642df1099af5.

VRR-29 | Discussion On Market Uniqueness Across Chains

Category	Severity	Location	Status
Logical Issue	Informational	RiskSteward/RiskStewardReceiver.sol (Update1): 59	Resolved

Description

The `lastProcessedUpdate` variable is utilized assuming that market addresses are unique across all chains. Currently, contracts are deployed with different addresses on each chain, ensuring uniqueness. However, if a uniform deploying address is used in future deployments across multiple chains, there is a possibility that markets on different chains could share the same address. This scenario could result in unintended interactions or tracking issues if the code does not distinguish between chains.

Recommendation

We recommend ensuring that deployment procedures will always be done in a way that ensures each market has a unique address.

Alleviation

[Venus, 12/29/2025]: We currently do not use factory contracts for market deployments, and there are no plans to adopt uniform or deterministic deployments across chains in the near future.

If we ever move toward uniform deployments in the future, we will explicitly take this consideration into account.

OPTIMIZATIONS | VENUS - RISK REWARD RECEIVER

ID	Title	Category	Severity	Status
VRR-01	Array Length Is Not Cached	Coding Issue	Optimization	Resolved
VRR-02	Cache Storage Variable Used Multiple Times	Gas Optimization	Optimization	Resolved
VRR-03	Use Of A String Instead Of Bytes32 Key In Mapping	Code Optimization	Optimization	Resolved

VRR-01 | Array Length Is Not Cached

Category	Severity	Location	Status
Coding Issue	Optimization	RiskSteward/RiskOracle.sol (Base): 390	Resolved

Description

Accessing a storage array's length on every iteration requires an additional storage read, which can increase gas consumption. Since the loop does not modify the array during iteration, the array's length can be read once before the loop starts and stored in a local variable. This cached value can then be used in the loop condition to optimize performance and reduce gas costs.

Recommendation

We recommend caching the array length in the lines cited above to save gas.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit ec4ff8ac0cfc5b1ac34f76f2d7e7d66dcaa6350e.

VRR-02 | Cache Storage Variable Used Multiple Times

Category	Severity	Location	Status
Gas Optimization	Optimization	RiskSteward/RiskOracle.sol (Base): 330, 338, 350, 353, 357	Resolved

Description

The `updateCounter` is incremented and the new value is then referenced multiple times in `_publishUpdate()`. The new value can instead be cached to save reading from storage multiple times.

Recommendation

We recommend caching storage variables that are repeatedly referenced to save gas.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 1865facd2ff2a2953dcfef07ff22913bf4c9e756.

VRR-03 | Use Of A String Instead Of Bytes32 Key In Mapping

Category	Severity	Location	Status
Code Optimization	Optimization	RiskSteward/RiskOracle.sol (Base): 18, 27	Resolved

Description

Using string as a key type in Solidity mappings is significantly more gas-intensive than using fixed-size types such as bytes32. Replacing string keys with bytes32 (or another fixed-size primitive) reduces gas costs for storage access, hashing, and comparisons, and results in more predictable and efficient execution.

Recommendation

We recommend replacing the use of the update type strings in mappings with their bytes32 updateTypeKey.

Alleviation

[CertiK, 12/24/2025]: The client made the recommended changes resolving this finding in commit 1865facd2ff2a2953dcfef07ff22913bf4c9e756.

APPENDIX | VENUS - RISK REWARD RECEIVER

Audit Scope

VenusProtocol/governance-contracts
RiskSteward/RiskOracle.sol
RiskSteward/MarketCapsRiskSteward.sol
RiskSteward/DestinationStewardReceiver.sol
RiskSteward/CollateralFactorsRiskSteward.sol
RiskSteward/IRMRiskSteward.sol
RiskSteward/RiskStewardReceiver.sol
RiskSteward/Interfaces/IRiskOracle.sol
RiskSteward/Interfaces/IRiskSteward.sol
RiskSteward/Interfaces/IRiskStewardReceiver.sol
interfaces/ICorePoolComptroller.sol
interfaces/ICorePoolVToken.sol
interfaces/IIsolatedPoolVToken.sol
interfaces/IIsolatedPoolsComptroller.sol

Finding Categories

Categories	Description
Gas Optimization	Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction on the total gas cost of a transaction.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.

Categories	Description
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Web3 Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.